

ChurchOS

Multi-Tenant Church Management ERP SaaS

Build Roadmap — Java + Spring Boot + IntelliJ IDEA + Claude Code

Prepared for: Engineer Ziba Valentine

August 2026

"ChurchOS" is used as a placeholder product name throughout this roadmap. Replace it with your actual brand.

Table of Contents

1. Vision & Product Positioning
2. Tech Stack Summary
3. Multi-Tenancy Architecture: Database-per-Tenant
4. Feature Set — Full Enterprise ERP Scope
5. IntelliJ IDEA: Step-by-Step Project Setup
6. Project Structure (Folder Tree)
7. Claude Code + IntelliJ Workflow
8. Build Roadmap — Phases & Milestones
9. Deployment on Contabo
10. Security, Compliance & Data Protection
11. Appendix — Recommended Libraries & Tools

(Open the Navigation Pane in Word — View → Navigation Pane — for clickable jump-to-section links built from the headings below.)

This roadmap is based on the architecture decisions you selected: **Spring Boot** as the backend framework, **database-per-tenant** isolation, **Contabo VPS (Docker-based, self-hosted)** as the deployment target, and a **full enterprise ERP feature scope** designed to compete with Planning Center, Breeze ChMS, ChurchTrac, Rock RMS, Tithe.ly, and Elvanto.

1. Vision & Product Positioning

ChurchOS is a SaaS platform that lets any church — from a 50-member congregation to a 10,000-member multi-campus network — sign up, get an isolated tenant environment in minutes, and run every operational function of the church from one system: people, money, events, volunteers, and communication.

To rival global players, ChurchOS needs three things most regional competitors lack:

True enterprise-grade multi-tenancy — each church's data is fully isolated (its own database), not just row-filtered in a shared table, which matters for financial and safeguarding data.

Fund accounting, not just "giving tracking" — most SMB-focused competitors (Breeze, ChurchTrac) stop at donation logging; Rock RMS and enterprise players go further with real ledgers. ChurchOS should match that.

Africa-aware payments and messaging alongside global rails — Stripe/PayPal for international reach, plus PayFast, Paystack, and Flutterwave for African markets, and SMS via Africa's Talking/Twilio.

2. Tech Stack Summary

Layer	Choice	Notes
Language / Runtime	Java 21 (LTS)	Virtual threads (Project Loom) help with I/O-heavy multi-tenant request handling
Framework	Spring Boot 3.x	Spring Data JPA, Spring Security, Spring Validation, Spring Cache
Build tool	Gradle (Kotlin DSL) or Maven	Gradle recommended for multi-module speed; Maven fine if you prefer XML familiarity
Database	PostgreSQL 16	One control-plane DB + one database per tenant
Migrations	Flyway	Separate migration sets for control-plane vs. tenant schemas
Caching / Sessions	Redis	Tenant resolution cache, rate limiting, session store
Async messaging	RabbitMQ (or Kafka later)	Email/SMS sending, webhooks, report generation, audit events
Auth	Spring Security + JWT (+ optional OIDC via Keycloak)	Role hierarchy: Super Admin → Org Admin → Staff → Volunteer → Member
API style	REST (OpenAPI/Swagger) + optional GraphQL later	REST first; mobile app and integrations consume the same API

Layer	Choice	Notes
Frontend (admin console)	React + TypeScript (Next.js), or Vaadin for pure-Java	React/Next.js recommended for a modern SaaS UI and easier hiring
Mobile	Flutter or React Native, consuming the REST API	Phase 3
Containerization	Docker + Docker Compose	Traefik as reverse proxy with automatic Let's Encrypt SSL
CI/CD	GitHub Actions → SSH/Docker deploy to Contabo	Build, test, containerize, push to registry, deploy
Monitoring	Prometheus + Grafana + Loki	Metrics, dashboards, log aggregation
Testing	JUnit 5, Mockito, Testcontainers	Testcontainers spins up real ephemeral Postgres per test run
IDE	IntelliJ IDEA Ultimate	Native Spring, Docker, and Database tooling
AI pair-programmer	Claude Code (CLI + JetBrains plugin)	Scaffolding, refactors, tests, code review inside IntelliJ

3. Multi-Tenancy Architecture: Database-per-Tenant

You chose database-per-tenant, the strongest isolation model — each church gets a fully separate PostgreSQL database. This is the same approach used by higher-end enterprise SaaS products and is the right call given churches handle sensitive data: giving records, safeguarding/child-protection information, and pastoral care notes.

3.1 Two-tier data model

Control-plane database (churchos_control): one database for the whole platform. Holds the tenant registry, subscription/billing plans, super-admin users, and platform-wide audit logs.

Tenant databases (tenant_<uuid> or tenant_<slug>): one per church, containing all operational data — members, giving, accounting, events, etc. Same schema across all tenants, kept in sync by Flyway migrations run against every tenant DB on deploy.

Control-Plane DB (churchos_control)

```
| - tenants (id, name, slug, subdomain, db_host, db_name, db_credentials_ref, plan, status)
| - subscriptions (tenant_id, plan_id, billing_cycle, stripe_customer_id, status)
| - plans (id, name, price, feature_flags, tenant_limit_users, storage_quota)
\ - platform_admins (super-admin accounts)
```

Tenant DB: tenant_grace_chapel

```
| - members, households, groups, tags
| - donations, pledges, campaigns, payment_methods
| - funds, accounts, ledger_entries, budgets
| - events, registrations, check_ins
| - volunteers, schedules
| - messages, campaigns, templates
\ - roles, users, audit_log (per-tenant staff accounts)
```

3.2 Tenant resolution (request routing)

Each church gets a subdomain (gracechapel.churchos.app) or custom domain. On every incoming request:

1. A `TenantResolverFilter` (a Spring `OncePerRequestFilter`) reads the `Host` header (or a custom `X-Tenant-ID` header for API/mobile clients).
2. It looks up the tenant in a Redis-backed cache (falling back to the control-plane DB on a cache miss) to get that tenant's database connection details.
3. It stores the resolved tenant identity in a `TenantContext` (a `ThreadLocal`, or a `ScopedValue` if you adopt Java 21 virtual threads) for the duration of the request.
4. It clears the context in a finally block so nothing leaks across requests/threads.

3.3 Dynamic DataSource routing

Because each tenant has its own physical database, you need Spring's `AbstractRoutingDataSource` backed by a lazily-built, size-bounded cache of `HikariDataSource` instances (e.g. a Caffeine cache keyed by tenant ID, with idle eviction so you don't hold thousands of open connection pools at once for tenants that aren't currently active).

```
public class TenantRoutingDataSource extends AbstractRoutingDataSource {
    @Override
    protected Object determineCurrentLookupKey() {
        return TenantContext.getCurrentTenant(); // returns tenant ID or null
    }
}
```

A `TenantDataSourceProvider` service resolves a tenant ID to a `DataSource`:

Cache hit → reuse the pooled `HikariDataSource`.

Cache miss → look up connection details in the control-plane DB, build a new `HikariDataSource` (small pool size per tenant, e.g. 2–5 connections), cache it, return it.

3.4 Tenant provisioning (new church signup)

A `TenantProvisioningService`, triggered when a new church completes signup or a super admin creates a tenant:

1. Insert a row into `tenants` (control-plane DB) with status `PROVISIONING`.
2. Programmatically `CREATE DATABASE tenant_<slug>` via an admin JDBC connection.
3. Run all tenant-schema Flyway migrations against the new database.
4. Seed default reference data: default fund ("General Fund"), default roles (Admin, Staff, Volunteer, Member), default membership statuses, a welcome email template.
5. Create the first admin user for that church and send an invite email.
6. Flip tenant status to `ACTIVE`.
7. Publish a `TenantProvisionedEvent` to RabbitMQ so other modules (billing, analytics) react asynchronously.

Keep this whole flow idempotent and wrapped so a failure at any step leaves the tenant in a clearly-flagged `FAILED` state rather than a half-created database — you'll want a retry/cleanup job for this.

3.5 Keeping tenant schemas in sync

Since every tenant has a separate physical schema, a migration written today must be applied to every tenant database, not just one. Two options:

Deploy-time migration runner — a startup task (or a dedicated Gradle/CI step) that iterates the tenant registry and runs Flyway against each tenant DB before the new application version starts serving traffic.

Flyway's built-in multi-schema support is not quite enough here since these are separate databases, not schemas within one database — so you'll write a small custom `TenantMigrationRunner` that loops over tenants and calls `Flyway.configure().dataSource(...).load().migrate()` per tenant.

At real scale (hundreds/thousands of tenants) this becomes the main operational cost of database-per-tenant — budget engineering time for it, and consider batching/parallelizing migrations.

4. Feature Set — Full Enterprise ERP Scope

This is the module list to rival Planning Center, Breeze, ChurchTrac, Rock RMS, and Tithe.ly combined. Build order is addressed in the Roadmap (Section 8); this section is the target scope.

4.1 Membership & CRM

People profiles with custom fields, households/families, membership status and history, visitor/first-time-guest tracking and follow-up pipelines, tagging and smart segments (dynamic lists like "givers who haven't given in 90 days"), group/ministry membership, background-check and safeguarding record tracking for children's ministry volunteers, a searchable online member directory (opt-in, privacy-controlled), and a self-service member portal for updating contact info and viewing giving history.

4.2 Giving & Online Payments

One-time and recurring online giving, pledge campaigns and capital campaigns with progress tracking, multiple funds per gift, text-to-give, kiosk/in-person giving stations, automatic tax-deductible giving statements (PDF, emailed at year-end), and a payment gateway abstraction layer supporting Stripe and PayPal (global) plus PayFast, Paystack, and Flutterwave (African markets) so a church can pick the processor available in its country.

4.3 Fund Accounting (the enterprise differentiator)

A real chart of accounts, multiple funds with restricted/unrestricted designations, double-entry ledger, budgets vs. actuals by fund/ministry, expense and reimbursement tracking with approval workflows, bank account reconciliation, and exportable financial statements (balance sheet, income statement, fund statement) — the feature area where most SMB competitors (Breeze, ChurchTrac) fall short of Rock RMS-tier platforms.

4.4 Events & Check-In

Event creation with registration and ticketing (paid or free), capacity limits and waitlists, secure children's check-in with printed name-tag/security-tag matching for pickup, kiosk mode (tablet-friendly), and attendance analytics per service/event/campus.

4.5 Small Groups & Discipleship

Group directories (life groups, Bible studies), group-level attendance and curriculum tracking, leader dashboards, and growth/discipleship pathway tracking for individual members.

4.6 Volunteers & Serving Teams

Position/role catalog per ministry, scheduling and shift assignment, automated reminder notifications, shift swap/self-service, blockout dates, and skills/availability matching.

4.7 Communications

Mass email campaigns with templates and segmentation, SMS/text messaging (Twilio globally, Africa's Talking for African numbers), push notifications for the companion mobile app, automated workflows (e.g. "send a welcome sequence to every new visitor"), and delivery/engagement analytics.

4.8 Multi-Campus / Multi-Site

Support for a single church organization running several physical (or online) campuses, with campus-level reporting roll-up into an organization-level dashboard, shared or campus-specific funds, and campus-scoped staff permissions — while still living inside that church's one tenant database.

4.9 Facilities & Resource Booking

Room and equipment scheduling, conflict detection, and integration with the events module.

4.10 Reporting & Business Intelligence

Prebuilt dashboards (giving trends, attendance trends, member growth, volunteer engagement), a custom report builder, scheduled report emails, and CSV/PDF export.

4.11 Workflow Automation

A trigger-action automation engine (e.g. "when a visitor attends twice, create a follow-up task for their campus pastor"; "when a pledge is 30 days overdue, send a reminder") — this is the kind of feature that separates enterprise ChMS platforms from basic ones.

4.12 Mobile App API & Member Portal

A documented REST API (OpenAPI/Swagger) powering both a staff-facing admin console and a member-facing mobile app: giving, event registration, group check-in, directory lookup, and push notifications.

4.13 Platform / Super-Admin Layer (the SaaS control plane itself)

Tenant management, subscription/plan management with usage-based feature gating (e.g. "Starter" plan caps at 200 members; "Enterprise" plan unlocks multi-campus and API access), in-app billing (Stripe Billing), platform-wide analytics, and a support/ticketing hook.

5. IntelliJ IDEA: Step-by-Step Project Setup

Step 1 — Install prerequisites

1. Install IntelliJ IDEA Ultimate (the Ultimate edition, not Community, because it has native Spring Boot run configurations, database tooling, and Docker integration — all of which this project needs).
2. Install JDK 21 (Temurin/Eclipse Adoptium build is a safe default). In IntelliJ: File → Project Structure → SDKs → Add JDK, or let IntelliJ download one for you.
3. Install Docker Desktop (or Docker Engine on Linux) — needed for local Postgres/Redis and for Testcontainers.
4. Install the Claude Code CLI on your machine, then install the Claude Code JetBrains plugin from Settings → Plugins → Marketplace → search "Claude Code". Sign in with your Anthropic account when prompted.

Step 2 — Create the project via Spring Initializr (inside IntelliJ)

1. File → New → Project → Spring Boot (IntelliJ's built-in Spring Initializr wizard).
2. Group: com.churchos — Artifact: church-erp — Name: church-erp — Type: Gradle - Kotlin (or Maven) — Language: Java — Java version: 21.
3. Dependencies to add at creation: Spring Web, Spring Data JPA, PostgreSQL Driver, Spring Security, Validation, Flyway Migration, Lombok, Spring Boot Actuator, Spring Boot DevTools.
4. Finish — IntelliJ generates the base project and imports it.

Step 3 — Configure the JDK and code style

1. File → Project Structure → Project — set Project SDK to JDK 21, language level 21.
2. Settings → Editor → Code Style → Java — import or agree on a formatter (Google Java Format or your own .editorconfig) so Claude Code's generated code matches your style automatically.
3. Enable annotation processing for Lombok: Settings → Build, Execution, Deployment → Compiler → Annotation Processors → Enable annotation processing.

Step 4 — Add the module/package skeleton

Rather than a true multi-module Gradle/Maven build (which adds ceremony you don't need yet), start as a modular monolith: one deployable Spring Boot application, internally organized into clearly bounded packages per business module (Section 6). This is faster to build and refactor early on, and you can extract any module into its own microservice later once you know your real scaling bottlenecks — extracting a well-isolated package is far easier than un-tangling a premature microservice mesh.

Step 5 — Local infrastructure with Docker Compose

Create `docker/docker-compose.dev.yml` with Postgres, Redis, RabbitMQ, and MailHog (for catching dev emails). Add an IntelliJ Run Configuration of type "Docker Compose" pointing at this file, and a Compound Run Configuration that starts the infra + the Spring Boot app together with one click.

Step 6 — Database tooling

Open IntelliJ's built-in Database tool window (View → Tool Windows → Database), add a PostgreSQL data source pointing at your local churchos_control database. IntelliJ Ultimate's DataGrip-based tooling gives you schema browsing, query console, and ER diagrams without leaving the IDE.

Step 7 — Version control

1. VCS → Enable Version Control Integration → Git, or git init from the terminal.
2. Add a .gitignore (IntelliJ can generate one for Java/Gradle projects) and commit the initial skeleton.
3. Push to a private GitHub/GitLab repository — this also unlocks Step 9's CI/CD.

Step 8 — First run

Run the generated ChurchErpApplication.main() from IntelliJ's gutter icon, confirm it starts against your local Docker Postgres, and hit the Actuator health endpoint (/actuator/health) to confirm it's alive.

Step 9 — Wire up Claude Code

Open the IntelliJ terminal (or the Claude Code panel from the plugin) inside the project root and run claude. Create a CLAUDE.md file at the project root (Section 7 explains exactly what to put in it) so every Claude Code session in this project has the right context automatically.

6. Project Structure (Folder Tree)

```
church-erp/
|-- build.gradle.kts / pom.xml
|-- settings.gradle.kts
|-- CLAUDE.md                                # Claude Code project instructions (Section
7)
|-- docker/
|   |-- docker-compose.dev.yml                # local Postgres, Redis, RabbitMQ, MailHog
|   |-- docker-compose.prod.yml              # production stack incl. Traefik
|   |-- Dockerfile                            # multi-stage build for the Spring Boot app
|   \-- traefik/traefik.yml
|-- .github/
|   \-- workflows/
|       |-- ci.yml                            # build + test on every PR
|       \-- deploy.yml                        # build image, push, deploy to Contabo
|-- db/
|   |-- control-plane/                        # Flyway migrations for churchos_control
|   |   \-- V1__init_control_plane.sql
|   \-- tenant/                              # Flyway migrations applied to every tenant
DB
|   |-- V1__init_core.sql
|   |-- V2__membership.sql
|   |-- V3__giving.sql
|   |-- V4__accounting.sql
|   |-- V5__events.sql
|   \-- V6__communications.sql
\-- src/
```

```

\-- main/
  |-- java/com/churchos/erp/
  |   |-- ChurchErpApplication.java
  |   |-- config/                                # SecurityConfig, DataSourceConfig,
RedisConfig, OpenApiConfig
  |   |-- tenant/                                # TenantContext, TenantResolverFilter,
TenantRoutingDataSource,
  |   |                                           #   TenantDataSourceProvider,
TenantProvisioningService
  |   |-- common/                                # BaseEntity, AuditableEntity, exceptions,
PageResponse, utils
  |   |-- platform/                              # super-admin: tenants, subscriptions,
plans, billing
  |   |   |-- membership/
  |   |   |   |-- domain/                        # Member, Household, Group, Tag
  |   |   |   |-- repository/
  |   |   |   |-- service/
  |   |   |   |-- dto/
  |   |   |   \-- web/                            # REST controllers
  |   |   |-- giving/
  |   |   |   |-- domain/                        # Donation, Pledge, Campaign, PaymentMethod
  |   |   |   |-- gateway/                       # StripeGatewayClient, PayFastGatewayClient,
PaystackGatewayClient...
  |   |   |   |-- service/
  |   |   |   \-- web/
  |   |   |-- accounting/
  |   |   |   |-- domain/                        # Fund, Account, LedgerEntry, Budget
  |   |   |   |-- service/
  |   |   |   \-- web/
  |   |   |-- events/
  |   |   |   |-- domain/                        # Event, Registration, CheckIn
  |   |   |   |-- service/
  |   |   |   \-- web/
  |   |   |-- groups/                            # small groups / discipleship
  |   |   |-- volunteers/                        # scheduling, serving teams
  |   |   |-- communications/
  |   |   |   |-- domain/                        # Campaign, MessageTemplate, MessageLog
  |   |   |   |-- channel/                       # EmailChannel, SmsChannel (Twilio, Africa's
Talking)
  |   |   |   \-- service/
  |   |   |-- reporting/                         # dashboards, custom report builder
  |   |   |-- automation/                       # workflow trigger/action engine
  |   |   \-- security/                         # JWT, roles/permissions, auth controllers
  \-- resources/
      |-- application.yml
      |-- application-dev.yml
      |-- application-staging.yml
      \-- application-prod.yml

```

Each business package (membership, giving, accounting, ...) is a self-contained module: its own domain, repository, service, and web sub-packages, and it only talks to other modules through their public service interfaces — never by reaching directly into another module's repositories. This discipline is what makes it realistic to peel a module out into its own microservice later without a rewrite.

7. Claude Code + IntelliJ Workflow

7.1 Set up project context once

Create CLAUDE.md at the repository root. This file is read automatically by Claude Code and should describe: the module boundaries above, the multi-tenancy rules ("never write a JPA query or repository method that isn't automatically scoped to TenantContext's active DataSource — there is no tenant_id column to filter by, isolation comes from the routing DataSource itself"), your coding conventions (Lombok usage, DTO/mapper pattern with MapStruct, test naming), and the commands to build/test/run (./gradlew build, ./gradlew test, docker compose -f docker/docker-compose.dev.yml up).

7.2 Division of labor

Use IntelliJ for what it's best at: structural navigation, refactoring (rename/extract/move with full compile-time safety), debugging with breakpoints, and the Database tool window for inspecting tenant data. Use Claude Code (via its JetBrains panel or the terminal) for what it's best at: scaffolding a new module's boilerplate (entity + repository + service + controller + DTO + mapper + Flyway migration in one pass), writing unit and integration tests against existing services, reviewing a diff before you commit, and drafting the OpenAPI documentation for new endpoints.

7.3 Suggested loop for building a new module

1. Describe the module to Claude Code in plain language (e.g. "add a Pledge Campaign feature to the giving module: entities, Flyway migration, service with create/list/close operations, REST controller, and unit tests").
2. Review the generated diff in IntelliJ's built-in diff viewer before accepting.
3. Run the generated tests locally (./gradlew test) — IntelliJ's test runner shows pass/fail inline.
4. Ask Claude Code to fix any failures, iterating in the same session rather than starting over.
5. Commit once green, with IntelliJ's Git integration or Claude Code's own commit flow.

7.4 Guardrails

Keep multi-tenancy-critical code (tenant/ package, TenantRoutingDataSource, provisioning logic) under stricter human review — treat AI-generated changes there as drafts to scrutinize line-by-line, since a bug in tenant isolation is the single highest-severity class of bug this product can have.

8. Build Roadmap — Phases & Milestones

Phase 0 — Foundation (Weeks 1-3)

Project skeleton in IntelliJ (Section 5), control-plane schema and tenant schema baseline migrations, TenantContext/TenantResolverFilter/TenantRoutingDataSource, tenant provisioning service with automated tests (using Testcontainers to actually create and migrate a throwaway Postgres database in CI), authentication and role-based authorization, Docker Compose dev environment, and CI pipeline (build + test on every PR).

Phase 1 — MVP Core (Weeks 4–10)

Membership & CRM module, basic giving module with one payment gateway integrated end-to-end (Stripe first, since it's the most documented — add PayFast/Paystack/Flutterwave once the pattern is proven), events module with registration (no check-in kiosk yet), email communications (transactional + basic campaigns), member self-service portal, and the super-admin tenant/subscription console. Deploy to a staging environment on Contabo. This is the version you can put in front of your first 3–5 pilot churches.

Phase 2 — Enterprise Differentiators (Weeks 11–18)

Fund accounting module (chart of accounts, ledger, budgets, financial statements), volunteer scheduling, small groups module, multi-campus support, additional payment gateways (PayFast, Paystack, Flutterwave) and SMS (Twilio, Africa's Talking), children's check-in with security tags, and facilities/resource booking. This phase is what closes the gap with Rock RMS-tier competitors.

Phase 3 — Scale & Intelligence (Weeks 19–26)

Reporting/BI dashboards and custom report builder, workflow automation engine, public REST API + API keys for third-party integrations, companion mobile app (Flutter/React Native) consuming the API, push notifications, and performance work on the tenant-routing layer (connection pool tuning, migration-runner parallelization) as tenant count grows.

Phase 4 — Go-to-Market Hardening (Ongoing)

Billing automation (Stripe Billing subscriptions, dunning, plan upgrades/downgrades), usage-based feature gating, in-app onboarding flows, SOC 2-style security hardening and audit logging, backup/disaster-recovery drills, and a public status page.

9. Deployment on Contabo

9.1 Server setup

Provision a Contabo VPS (an 8 vCPU / 24–32 GB RAM Cloud VPS tier is a reasonable starting point for a self-hosted multi-tenant Postgres fleet; scale up or split app/DB across two VPS instances as tenant count grows). Install Docker Engine + Docker Compose plugin, and lock down SSH (key-only auth, non-root user, ufw firewall allowing only 22/80/443).

9.2 Reverse proxy & wildcard SSL

Run Traefik as the reverse proxy in front of the Spring Boot app container. Configure a wildcard DNS record (*.churchos.app → your Contabo IP) and Traefik's Let's Encrypt DNS challenge (via your DNS provider's API) to auto-issue a wildcard TLS certificate covering every tenant subdomain without manual renewal.

9.3 Database hosting strategy

Run PostgreSQL in a Docker container (or as a native service) on the same VPS to start, with each tenant as a separate database on that one Postgres instance/cluster. As you grow, this is the natural place to introduce tiering: lower-cost plans stay on the shared Postgres instance, while an "Enterprise" plan tenant gets its own dedicated Postgres container/VPS — a feature you can market

directly against competitors, since few SMB-focused ChMS platforms offer dedicated database isolation as a plan tier.

9.4 CI/CD pipeline

GitHub Actions workflow: on push to main — run tests → build the Docker image → push to a container registry (GitHub Container Registry or Contabo-compatible private registry) → SSH into the Contabo VPS → docker compose pull && docker compose up -d → run the tenant migration runner → smoke-test the health endpoint. Keep a manual approval gate on the deploy job until you're confident in the pipeline.

9.5 Backups

Nightly pg_dump per tenant database (or WAL-based continuous backup for larger tenants), shipped to Contabo Object Storage or an S3-compatible bucket, with a documented and tested restore procedure — a backup you haven't restored from is not a backup.

9.6 Monitoring

Prometheus scraping Spring Boot Actuator's /actuator/prometheus endpoint, Grafana dashboards for request latency/error rate/DB pool saturation, and Loki for centralized log search across containers. Alert on tenant provisioning failures and on connection-pool exhaustion specifically, since those are the failure modes unique to this architecture.

10. Security, Compliance & Data Protection

Church membership data includes sensitive categories (religious affiliation is itself sensitive personal data under most privacy laws, plus safeguarding/child-protection records and financial giving history), so build compliance in from the start rather than retrofitting it: encrypt data at rest (Postgres disk encryption or filesystem-level) and in transit (TLS everywhere, including between the app and each tenant database), implement per-tenant audit logging of who accessed or changed what, support data export and account/data deletion requests (both POPIA, South Africa's data protection law, and GDPR if you'll have European members, give data subjects these rights), and enforce strong password/MFA policies for staff accounts given they can access financial and child-safety data.

11. Appendix — Recommended Libraries & Tools

Purpose	Library
Object mapping (entity ↔ DTO)	MapStruct
API documentation	springdoc-openapi
PDF generation (giving statements, reports)	OpenPDF or Apache PDFBox
Excel/CSV export	Apache POI
Payment gateways	Stripe Java SDK, PayFast REST integration, Paystack Java SDK, Flutterwave

Purpose	Library
SMS	Twilio SDK (global), Africa's Talking SDK (Africa)
Email	Spring Mail + a transactional provider (SendGrid, Postmark, or Amazon SES)
Feature flags / plan gating	Custom (plans.feature_flags JSON column), or Unleash
Testing	JUnit 5, Mockito, Testcontainers, RestAssured for API tests
Code quality	Checkstyle/SpotBugs, SonarQube (self-hosted or SonarCloud)

Next Step

Start with Phase 0. Once the tenant provisioning flow and the routing DataSource are working and covered by integration tests (this is the highest-risk, hardest-to-retrofit part of the whole system), the rest of the modules follow a repeatable pattern that Claude Code can help you scaffold quickly inside IntelliJ.